Attendance Management System

UML Design and Class Description · COMP 301 Advanced Programming (Java) · Fall 2026-2027

1. Project description

The system helps a college keep track of the students who attend each class. The administrator registers the students, the teachers and the courses. A teacher logs in with his email, chooses one of his courses and the kind of class (**Lecture**, **Lab** or **Section**), then types the attendance ID that every student says. When the class ends, every enrolled student who was not marked is saved as absent. The system produces attendance reports for each course and each session. It is written in Java with a Swing GUI, and its data is stored in text files.

2. Requirements covered by the design

Requirement	How the design meets it
Admin, Teacher and Student are persons	All three extend the abstract class `Person` (name, email).
Random unique IDs	Student `studentAttendanceId` : 4 digits; Teacher `employeeId` : 5 digits; Admin `adminId` : 6 digits. Generated by `IdGenerator` , which remembers every used ID.
Admin creates students, teachers and courses	`Admin.createStudent / createTeacher / createCourse` , plus `assignTeacher` and `enrollStudent` .
Attendance is split into Lecture, Lab and Section	Abstract class `AttendanceSession` with three subclasses: `LectureSession` , `LabSession` , `SectionSession` .
Teacher flow: login → course → type → IDs → end	`Teacher.startSession` , `markAttendance` , `endSession` ; GUI in `TeacherFrame` .
Only Admin and Teacher log in	Only they implement the interface `Authenticatable` .
Attendance reports	`AttendanceReport.courseReport` and `sessionReport` .
Data kept after closing the program	`DataStore` saves and loads five text files (the project database).

3. What changed from the previous UML

The previous diagram was correct in idea but too large, and its lines crossed each other. The new design keeps every essential part and removes what the program does not need:

- The enumerations `AttendanceStatus` (PRESENT/ABSENT/LATE/EXCUSED) and `SessionType` were removed. A record now has `present: boolean` , and the type of a session comes from the overridden method `getType()` (polymorphism).
- `DataStore` is no longer a Singleton; it uses `static` members (Lecture 2), which is simpler.
- `Teacher` and `Student` no longer keep their own course lists. `Course` is the only owner of "who teaches" and "who is enrolled", so the data cannot become inconsistent.
- `AttendanceReport` keeps no data; it only has static methods that build the report text. The `Map` , `LocalTime` , remarks and phone number were removed.
- All lines are now straight and orthogonal, and the model is split into readable diagrams.

4. Architecture and packages

The code is divided into three packages. **model** contains the classes of the problem (people, courses, sessions). **data** keeps all objects in `ArrayList`s while the program runs and saves them in the folder `data/`. **gui** contains the Swing windows; it only reads input, calls the model and shows the results. `Main` loads the data and opens the login window.

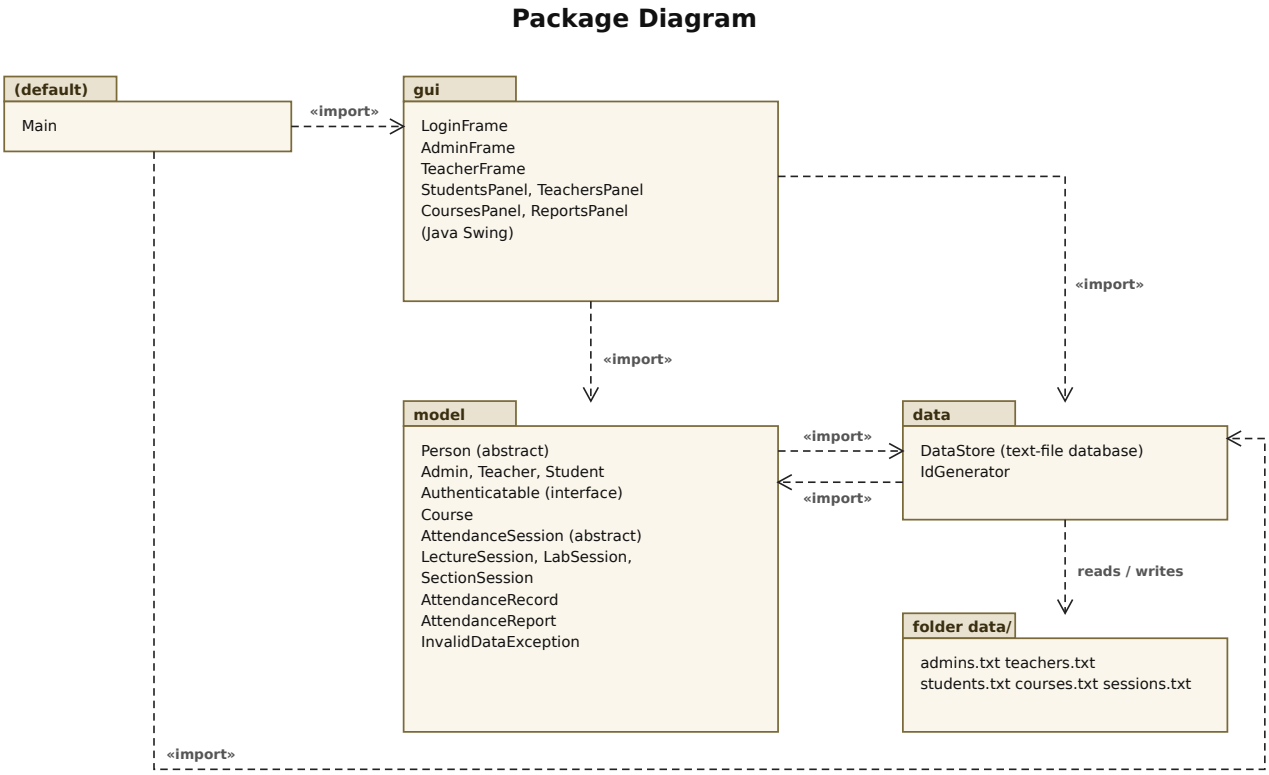


Figure 1. Package diagram

UML notation used

Line	Meaning in this project
Solid line, hollow triangle	Inheritance (`extends`), e.g. Teacher → Person
Dashed line, hollow triangle	Realization (`implements`), e.g. Admin → Authenticatable
Filled diamond	Composition: the part cannot live without the whole (Course ◊ AttendanceSession)
Hollow diamond	Aggregation: the part lives independently (Course ◊ Student)
Solid line, open arrow	Association, the arrow shows which class holds the reference
Dashed line, open arrow	Dependency («uses», «create», «import»)
Italic / underlined	Abstract class or method / static member

Class Diagram 1 - Domain Model (package model)

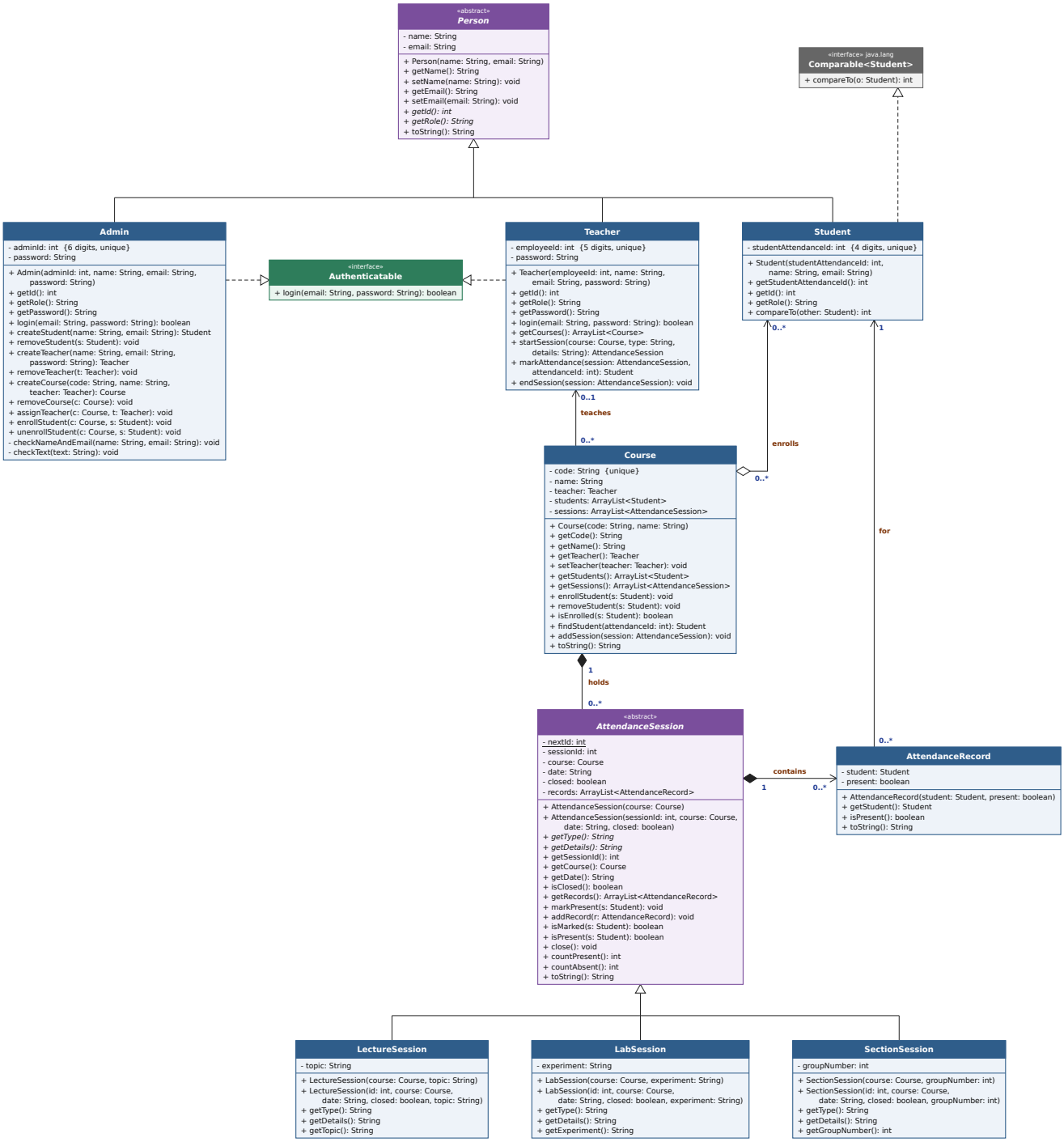


Figure 2. Class diagram 1: the domain model (package `model`)

Class Diagram 2 - Data, IDs, Reports and Exceptions (packages data + model)

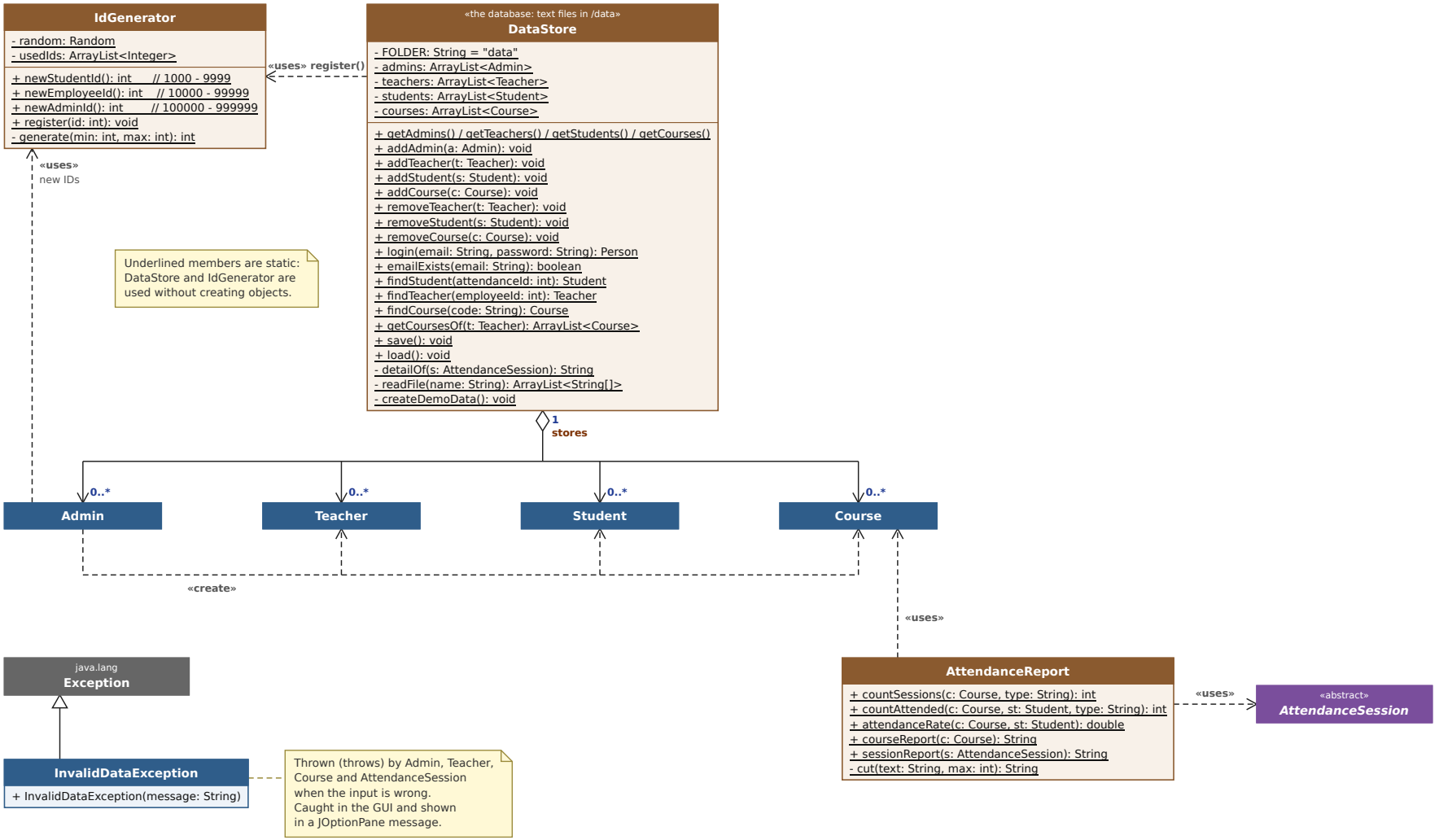


Figure 3. Class diagram 2: database, ID generation, reports and the custom exception

The four lists inside `DataStore` hold every Admin, Teacher, Student and Course. Each change made through the GUI calls `DataStore.save()`, which rewrites the text files. On start-up `DataStore.load()` reads them back and registers every ID in `IdGenerator`, so a new random ID can never repeat an old one.

Class Diagram 3 - GUI (package gui, Java Swing)

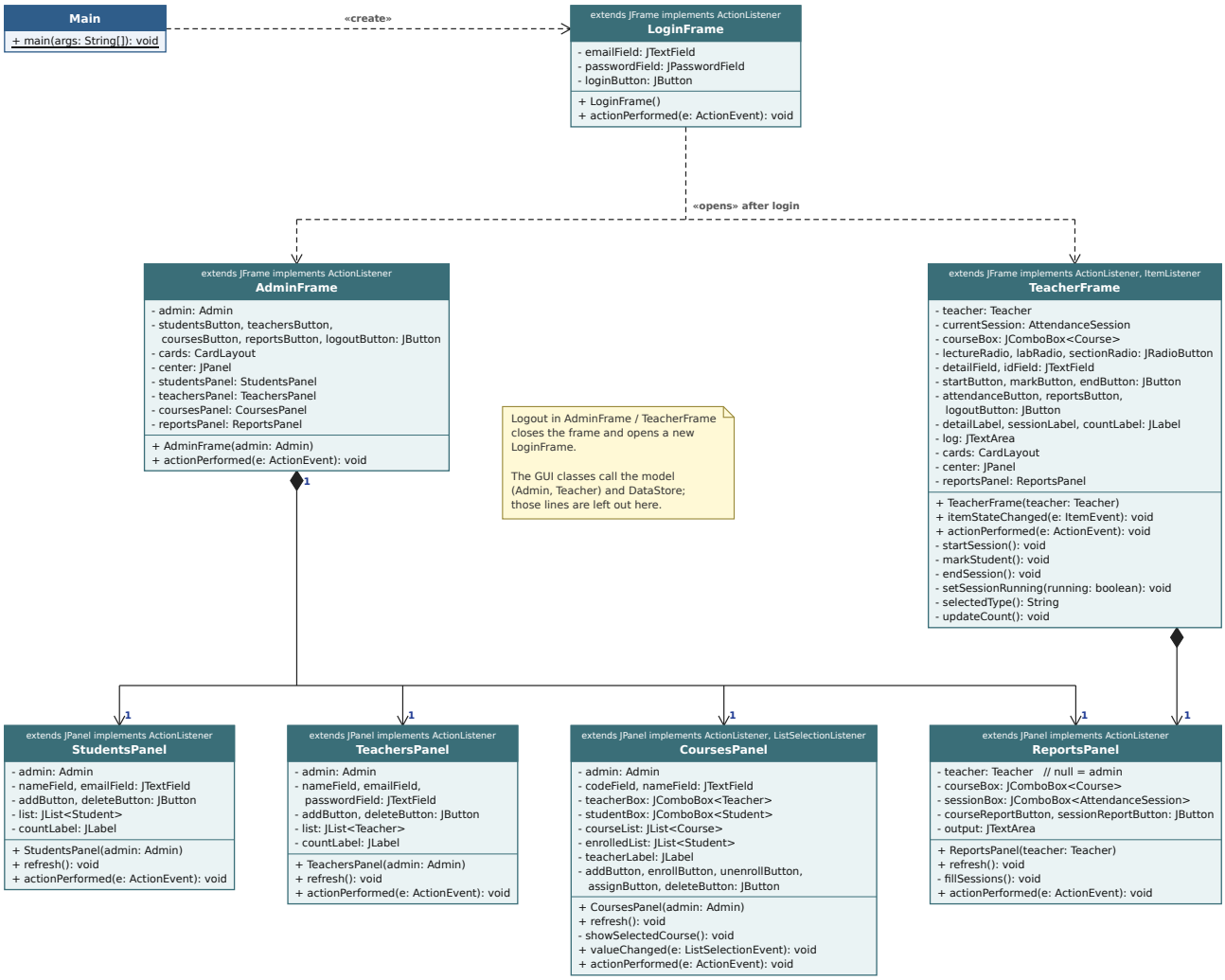


Figure 4. Class diagram 3: the GUI (package `gui`)

Every window `extends JFrame` and every panel `extends JPanel`. Event handling follows Lecture 8: the class itself implements the listener interface and uses `e.getSource()` to know which control fired the event. `ReportsPanel` is shared: the admin sees all courses, a teacher sees only his own.

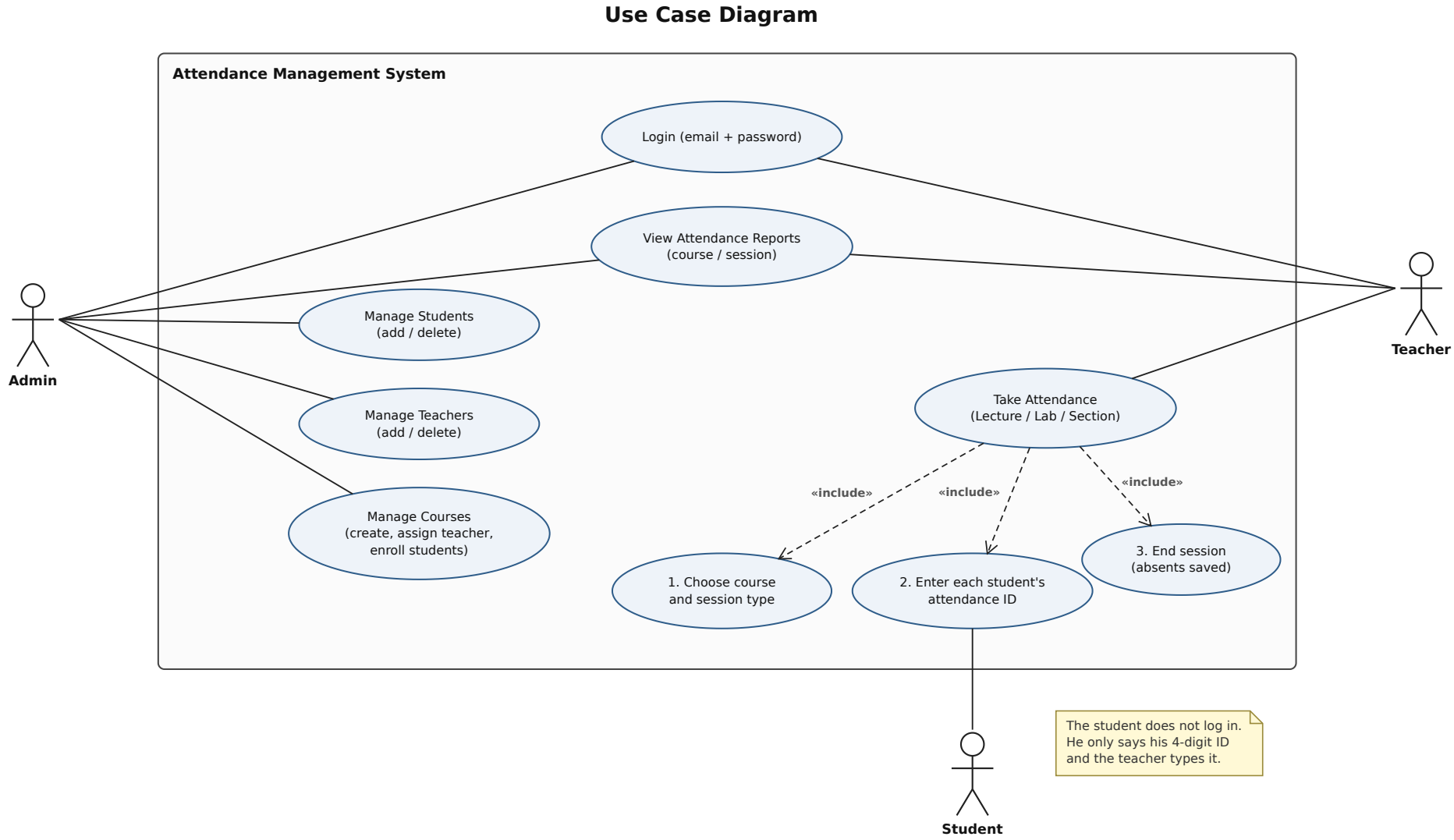


Figure 5. Use case diagram

The admin manages the data; the teacher takes attendance. The three steps of taking attendance are separate use cases that "Take Attendance" always includes. The student is a secondary actor: he never logs in, he only gives his 4-digit ID.

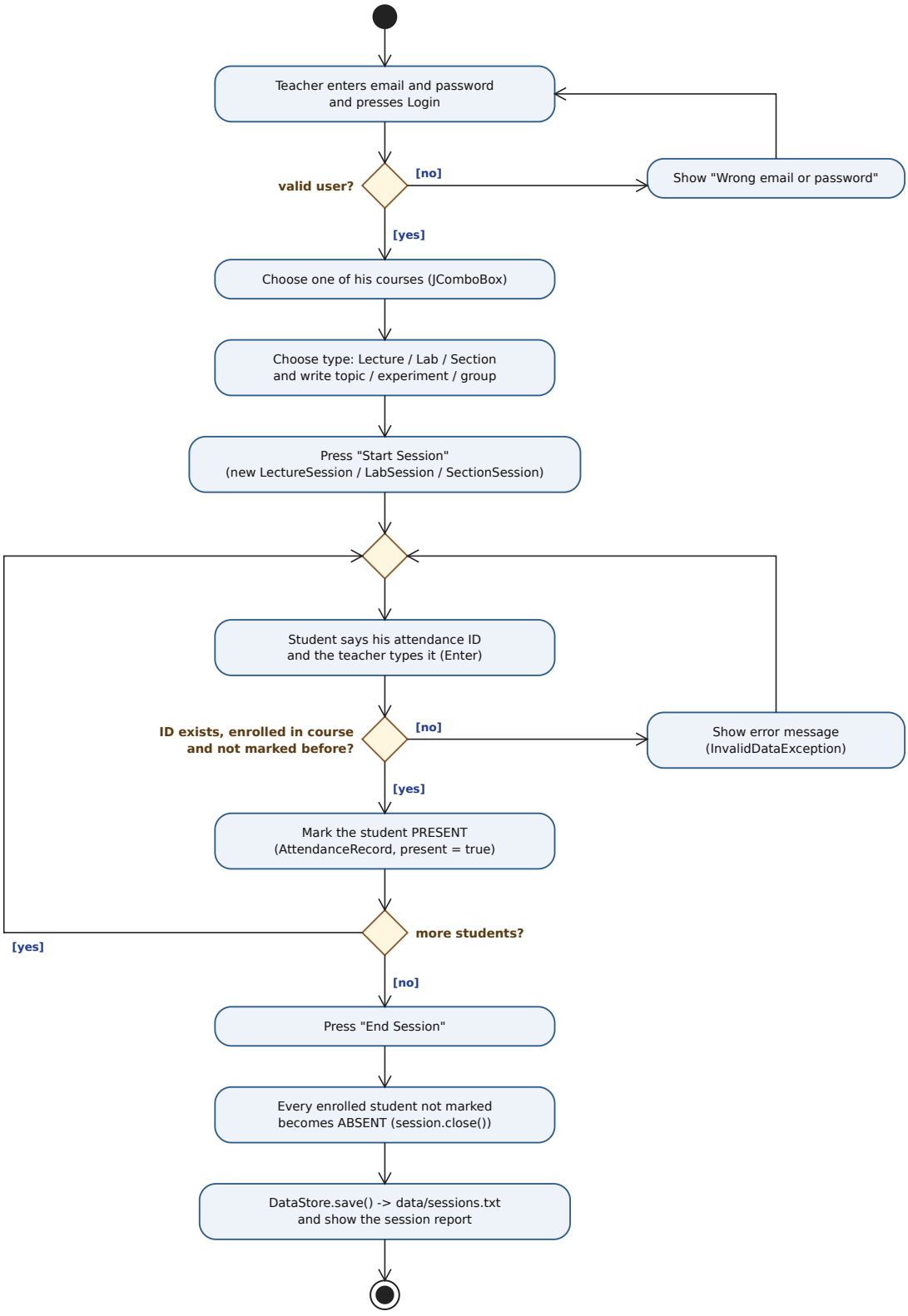


Figure 6. Activity diagram of the teacher's attendance flow

5. Description of the classes

Class	Kind	Responsibility
`Person`	abstract class	Common data of every person (name, email). Declares the abstract methods `getId()` and `getRole()` that every subclass overrides.
`Authenticatable`	interface	Contract for users who can log in: `login(email, password)`. Implemented by Admin and Teacher only.
`Admin`	class	6-digit `adminId`. Creates and removes students, teachers and courses, assigns a teacher to a course and enrolls students. Validates the input and throws `InvalidDataException`.
`Teacher`	class	5-digit `employeeId`. Gets his courses, starts a session of the chosen type, marks students by attendance ID and ends the session.
`Student`	class	4-digit `studentAttendanceId`. Does not log in. Implements `Comparable<Student>` so lists are sorted by name.
`Course`	class	Unique code and name, its teacher, the enrolled students and all its sessions. Finds an enrolled student by attendance ID.
`AttendanceSession`	abstract class	One attendance sheet: ID, course, date, open/closed state and a list of records. Marks students present, refuses duplicates, and on `close()` adds an absent record for every unmarked student.
`LectureSession`, `LabSession`, `SectionSession`	subclasses	Override `getType()` and `getDetails()`; each adds its own attribute: topic, experiment title, group number.
`AttendanceRecord`	class	One line of a sheet: a student and whether he was present.
`AttendanceReport`	utility (static)	Counts sessions per type, sessions attended, the attendance percentage, and builds the course and session reports as text.
`InvalidDataException`	checked exception	Custom exception (Lecture 6) carrying a clear message for any wrong input; the GUI shows it in a dialog.
`DataStore`	utility (static)	The database. Keeps the lists, searches them (login, find by ID/code, email exists) and saves/loads the text files.
`IdGenerator`	utility (static)	Produces random IDs in the right range and keeps a list of used IDs to guarantee uniqueness.
`LoginFrame`	JFrame	Email and password form; opens `AdminFrame` or `TeacherFrame`.
`AdminFrame`	JFrame	Menu on the left and a `CardLayout` that shows one of four panels.
`StudentsPanel`, `TeachersPanel`	JPanel	Add form, list and delete button; shows the generated ID after adding.
`CoursesPanel`	JPanel	Create a course, assign its teacher, enroll or remove students, delete a course.
`ReportsPanel`	JPanel	Chooses a course or a session and prints its report in a text area.
`TeacherFrame`	JFrame	Step 1: course and type (radio buttons); step 2: attendance IDs; End Session; plus a reports card.

`Main`	class	Program entry: `DataStore.load()` then `new LoginFrame()`.

6. Relationships and multiplicities

Relationship	Explanation
Course → Teacher, 0..* to 0..1	A course has at most one teacher (none until the admin assigns one); a teacher may teach many courses.
Course ◊→ Student, 0..* to 0..* (aggregation)	A student can be in many courses and a course has many students; deleting a course does not delete its students.
Course ◆ AttendanceSession, 1 to 0..* (composition)	A session belongs to exactly one course and is deleted with it.
AttendanceSession ◆→ AttendanceRecord, 1 to 0..*	A record exists only inside its session.
AttendanceRecord → Student, 0..* to 1	Every record is about one student.
DataStore ◊ Admin / Teacher / Student / Course	The store keeps the objects but does not create their meaning.

7. Object-oriented concepts applied

Concept (lecture)	Where it is used
Encapsulation (1)	All attributes are `private` and accessed through getters/setters.
static members (2)	`DataStore`, `IdGenerator`, `AttendanceReport`, and the counter `AttendanceSession.nextId`.
ArrayList and generics (3)	Every collection in the project, e.g. `ArrayList<Student>`.
Inheritance, overriding, polymorphism (4)	Person hierarchy and session hierarchy; `getType()` returns a different value for each session object.
Abstract classes and interfaces (5)	`Person`, `AttendanceSession`, `Authenticatable`, `Comparable<Student>`.
Exceptions and files (6)	Custom checked `InvalidDataException`; `BufferedReader/BufferedWriter` for the database.
Swing, layouts, events (7, 8)	JFrame, JPanel, BorderLayout, FlowLayout, GridLayout, CardLayout; ActionListener, ItemListener, ListSelectionListener.